

01 类对象属性与方法

目标：闭卷能写类、对象、`self`、`__init__`、属性、方法、`__str__`、私有属性，并能完成实验报告中的圆类和课程类。

总主线

类是模板

- 类名(...) 创建对象
- 参数传给 `__init__`
- `self` 表示当前对象
- `self.属性` 保存对象状态
- 对象.方法() 操作对象状态

类和对象【必须闭卷会写】

1. 它解决什么问题

当一组数据和操作总是绑定在一起时，用类更合适。例如学生有姓名和成绩，圆有圆心、半径、颜色，还能计算面积和周长。

2. 最小语法

```
class Student:
    pass

student = Student()
print(student)
```

3. 对象创建和调用过程

`Student` 是类
`student` 是对象或实例
`Student()` 是创建对象

4. 常见错误

```
student = Student
```

这只是把类本身赋值给变量，没有创建对象。创建对象必须写括号： `Student()`。

5. 考试可能怎么出

让你定义 `Student`、`Circle`、`Course`、`Book` 等类并创建对象测试。

6. 最小闭卷题

定义一个 `Student` 类，创建对象并给对象动态添加 `name` 属性。

self【必须闭卷会写】

1. 它解决什么问题

`self` 表示“当前这个对象自己”。同一个类可以创建很多对象，方法里必须知道现在操作的是哪一个对象。

2. 最小语法

```
class Student:
    def show(self):
        print(self)

student = Student()
student.show()
```

3. 对象创建和调用过程

```
student.show()
```

- Python 自动把 `student` 传给 `self`
- `show(self)` 里的 `self` 就是 `student`

4. 常见错误

```
class Student:
    def show():
        print("hello")
```

```
Student().show()
```

错误原因：实例方法漏写 `self`，对象调用时 Python 会自动传一个对象进去，参数数量对不上。

5. 考试可能怎么出

改错题：方法定义漏写 `self`。

6. 最小闭卷题

定义 `Dog` 类，写 `bark(self)` 方法，创建对象后调用。

构造方法 `__init__` 【必须闭卷会写】

1. 它解决什么问题

创建对象时，把必要数据一次性保存到对象里。

2. 最小语法

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
```

```
student = Student("Tom", 80)
print(student.name)
print(student.score)
```

3. 对象创建和调用过程

```
Student("Tom", 80)
→ 自动调用 __init__(self, "Tom", 80)
→ self.name = "Tom"
→ self.score = 80
→ 数据保存在对象里
```

`name` 和 `self.name` 的区别：

`name`: 形参, 只是 `__init__` 里的临时变量
`self.name`: 实例属性, 保存在对象中

4. 常见错误

```
class Student:
    def __init__(self, name):
        name = name
```

错误原因: 左边的 `name` 还是局部变量, 没有写成 `self.name`, 对象里没有保存属性。

5. 考试可能怎么出

要求创建对象时传入属性, 例如 `Circle(0, 0, 5, "red")`。

6. 最小闭卷题

写一个 `Car` 类, 创建对象时传入 `color`, 并能输出颜色。

实例属性与实例方法: 圆类【必须闭卷会写】

1. 它解决什么问题

实验报告题: 设计圆类, 包含圆心位置、半径、颜色, 能计算周长和面积。

2. 最小语法

```
import math

class Circle:
    def __init__(self, x, y, radius, color):
        self.x = x
        self.y = y
        self.radius = radius
        self.color = color

    def circumference(self):
        return 2 * math.pi * self.radius

    def area(self):
        return math.pi * self.radius * self.radius
```

```
def show_info(self):
    print(f"圆心: ({self.x}, {self.y})")
    print(f"半径: {self.radius}")
    print(f"颜色: {self.color}")
    print(f"周长: {self.circumference():.2f}")
    print(f"面积: {self.area():.2f}")

circle = Circle(0, 0, 5, "红色")
circle.show_info()
```

3. 对象创建和调用过程

```
Circle(0, 0, 5, "红色")
→ self.x/self.y/self.radius/self.color 保存状态
→ circle.area() 读取 self.radius 计算面积
→ circle.show_info() 调用对象自己的方法
```

4. 常见错误

```
def area(self):
    print(math.pi * self.radius * self.radius)
```

如果后续还要继续使用面积，应该 `return`，不是只 `print`。

5. 考试可能怎么出

闭卷写 `Circle`，并测试周长和面积。

6. 最小闭卷题

定义 `Rectangle` 类，保存 `width/height`，返回面积和周长。

类属性与实例属性【必须会读会改】

1. 它解决什么问题

有些数据属于所有对象，例如学校名、车轮数量；有些数据属于单个对象，例如姓名、颜色。

2. 最小语法

```
class Student:
    school = "江西农业大学"

    def __init__(self, name):
        self.name = name

s1 = Student("Tom")
s2 = Student("Jerry")

print(s1.school)
print(s2.school)
print(s1.name)
```

3. 对象创建和调用过程

`school` 在类上，多个对象共享
`name` 在对象上，每个对象各自保存

4. 常见错误

```
s1.school = "其他学校"
```

这会在 `s1` 对象上创建同名实例属性，遮蔽类属性，不是修改所有对象的学校名。

5. 考试可能怎么出

程序输出题：类属性被对象同名属性遮蔽后输出什么。

6. 最小闭卷题

定义 `Car.wheels = 4`，创建两个对象，给其中一个对象设置 `wheels = 3`，预测输出。

三种方法【必须会读会改】

1. 它解决什么问题

不同方法接收的“第一个东西”不一样：对象、类，或者什么都不自动接收。

2. 最小语法

```
class Example:
    count = 0

    def instance_method(self):
        return "实例方法"

    @classmethod
    def class_method(cls):
        return cls.count

    @staticmethod
    def static_method(score):
        return score >= 60
```

3. 对象创建和调用过程

`self` 接收当前对象
`cls` 接收当前类
`staticmethod` 不自动接收对象或类

4. 常见错误

在静态方法里直接使用 `self`：

```
@staticmethod
def is_pass():
    return self.score >= 60
```

静态方法没有 `self`。

5. 考试可能怎么出

阅读题：判断实例方法、类方法、静态方法分别能访问什么。

6. 最小闭卷题

写一个 `ScoreTool`，静态方法判断成绩是否及格。

`__str__` 与 `__repr__` 【必须闭卷会写 / 了解即可】

1. 它解决什么问题

控制 `print(对象)` 显示什么。

2. 最小语法

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __str__(self):
        return f"{self.name}: {self.score}"

student = Student("Tom", 80)
print(student)
```

3. 对象创建和调用过程

```
print(student)
→ Python 尝试调用 student.__str__()
→ __str__ 返回字符串
```

4. 常见错误

```
def __str__(self):
    print(self.name)
```

`__str__` 必须返回字符串，不能只在里面 `print`。

5. 考试可能怎么出

改错题：`__str__` 返回了非字符串或没有返回。

6. 最小闭卷题

定义 `Book`，打印对象时显示书名和价格。

`__repr__` 了解即可：

```
__str__ 面向用户
__repr__ 面向开发和调试
```

私有属性与课程类【必须闭卷会写】

1. 它解决什么问题

实验报告题：课程类包含课程名、编号、老师、上课地点，其中地点是私有属性。

2. 最小语法

```
class Course:
    def __init__(self, name, number, teacher, location):
        self.name = name
        self.number = number
        self.teacher = teacher
        self.__location = location

    def get_location(self):
        return self.__location

    def show_info(self):
        print(f"课程名: {self.name}")
        print(f"课程编号: {self.number}")
        print(f"任课老师: {self.teacher}")
        print(f"上课地点: {self.__location}")
```

3. 对象创建和调用过程

```
Course(...)
```

- `__location` 被保存为私有属性
- 外部不要直接 `course.__location`
- 通过 `get_location()` 或 `show_info()` 访问

4. 常见错误

```
course = Course("Python", "PY001", "王老师", "A101")
print(course.__location)
```

会报 `AttributeError`。双下划线会发生名称改写，不是绝对安全，但外部应通过方法访问。

5. @property 最小版本【必须会读会改】

```
class Course:
    def __init__(self, name, location):
        self.name = name
```

```
        self.__location = location

    @property
    def location(self):
        return self.__location

course = Course("Python", "A101")
print(course.location)
```

6. 最小闭卷题

定义 `Course` 类，`location` 私有，通过 `get_location()` 返回地点。

对象列表【必须会读会改】

1. 它解决什么问题

多个对象通常放进列表，例如多个学生。

2. 最小语法

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def show(self):
        return f"{self.name}:{self.score}"

students = [
    Student("A", 80),
    Student("B", 90),
]

for student in students:
    print(student.show())

top = max(students, key=lambda student: student.score)
print(top.name)
```

3. 常见错误

```
top = max(students)
```

对象默认不知道按什么比较，要写 `key=lambda student: student.score`。

类与对象执行流程图



属性与方法对照表

内容	写法	保存在哪里	作用
实例属性	<code>self.name</code>	对象	保存对象状态
类属性	<code>Student.school</code>	类	所有对象共享
实例方法	<code>def show(self)</code>	类中定义，对象调用	操作对象
类方法	<code>@classmethod</code>	类中定义	操作类
静态方法	<code>@staticmethod</code>	类中定义	工具函数

10 个易错点

- 1. 创建对象要写 `类名()`。
- 2. 实例方法第一个参数必须写 `self`。
- 3. `self.name = name` 才是保存到对象。
- 4. `__init__` 前后都是双下划线。
- 5. 创建对象传参数量要和 `__init__` 匹配。
- 6. 类属性可能被对象同名属性遮蔽。
- 7. `__str__` 必须返回字符串。

8. 私有属性不能直接用 `obj.__attr` 访问。
9. 静态方法没有 `self`。
10. 对象列表排序要指定 `key`。

8 个口头自测

1. 类和对象分别是什么？
2. `Student()` 会触发哪个方法？
3. `self` 当前指向谁？
4. `name` 和 `self.name` 有什么区别？
5. 类属性和实例属性有什么区别？
6. `print(obj)` 会优先调用什么？
7. 双下划线私有属性为什么外部不能直接访问？
8. 对象列表按成绩排序怎么写？

5 个最小闭卷类题

1. 写 `Student(name, score)`，包含 `show()`。
2. 写 `Circle(x, y, radius, color)`，返回面积和周长。
3. 写 `Course(name, number, teacher, location)`，`location` 私有。
4. 写 `Book(title, price)`，实现 `__str__`。
5. 写对象列表，找最高分学生。

答案区

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def show(self):
        return f"{self.name}:{self.score}"

students = [Student("A", 80), Student("B", 90)]
top = max(students, key=lambda student: student.score)
print(top.show())
```

[继续学习继承封装多态](#)